Temco Platform

The registry the module reads · built Nov 10, 2026

The registry the module reads

`docs/help/registry.json` in each product repository is the single description of that product's documentation set. This extension is a **reader** of it. The schema is owned by the generator in the product repo (`scripts/docs-registry.mjs`) — if a field ever needs to change, it changes there first and this reader follows.

Shape

JSON

```
{
  "version": 1,
  "areas": {
    "t3000": { "title": "t3000", "audience": "user", "noindex": false }
  },
  "pages": {
    "t3000/api-reference/data-points": {
      "title": "Data Points API",
      "area": "t3000",
      "chapter": "api-reference",
      "order": 20,
      "audience": "user",
      "ship": true,
      "owner": "docs",
      "tags": [],
      "keywords": [],
      "help_ids": [],
      "code": [],
      "reviewed": null
    }
  }
}
```

Fields, and what they decide

Field	Where	Meaning
`areas`	registry	the top level of the site: one entry per product area, with its default audience and whether it is excluded from the published site
`pages`	registry	every page, keyed by **page id**
`title`	page	the page's name, used for search, navigation and the tree
`area` , `chapter`	page	where the page sits; both must agree with the page id
`order`	page	position inside its chapter. Pages are ordered by it, then by title
`audience`	page	`user` , `developer` or `contributing` ; falls back to the area's audience
`ship`	page	`false` keeps the page out of the shipped help build
`owner`	page	who is answerable for the page's accuracy
`tags` , `keywords` , `help_ids` , `code`	page	search facets, context-help ids, and the code globs a change should make the page stale. Empty is normal
`reviewed`	page	ISO date of the last human review, or `null` . `null` is a **count**, not a warning

Page id → file

The page id is the path under `docs/` , without the extension:

Page id	File that must exist
`t3000/api-reference/data-points`	`docs/t3000/api-reference/data-points.md`

That single rule is what makes a page "missing" checkable, and it is why this extension never guesses at paths: the id is the path.

Generated neighbours

The product repo also emits, and this extension may read:

- `docs/help/index.json` — per page: its headings, the last commit that touched it (author, sha, date), and the generator's `stale` verdict.
- `docs/help/page-meta.json` — the same provenance in the smaller shape the published page reads.

Neither is required. Absent, the extension works exactly as before; only provenance goes missing. A malformed one is a **warning on that file** — the fix is to regenerate it in the product repo — never an error, because it hides nothing else.

What the extension shows from them: a page's last change and headings in its tooltip, a `stale` marker on the page (description and icon), and a stale count in the repository and area tooltips plus the status report. The page's `code` globs are read from the same place, because that is what a `Draft update`'s commit range is read against.

`stale` is read, never computed here. The verdict — a commit touched a path the page claims in `code`, after its last review — belongs to the product repo's generator (`05-content-and-registry.md`, *Staleness*). Two answers to that question would be worse than none, so the extension only repeats the generator's answer.

Structural checks

Run on every refresh, cheap enough to run on 335 pages without thinking about it:

Check	Severity	Reported on
the page has no title	error	the page
`area` is not declared in `areas{}`	error	the page
the id disagrees with `area` / `chapter`	error	the page
registered, but the file does not exist	error	`registry.json`
no `order`	warning	the page
two pages share one title	warning	both pages
two pages share one `order` inside a chapter	warning	both pages
the registry declares no pages	warning	`registry.json`
the registry is not valid JSON	error	`registry.json`

Deliberately not checked here: prose quality, links, images and staleness. Those belong to the product repo's own gates (`npm run docs:verify`, the media audit, `docs-owed`) and to the AI phase, where a commit range can be read. Duplicating them in the extension would create two answers to one question.

The feedback file — a proposal, not a contract

Where a feedback row comes from is not settled: the feedback design (`09-feedback-subscriptions-notifications.md`) assumes an endpoint on the T3000 server, while the management overview (`16-`

`overview.md`) describes a different hosting model. This module reads the smallest shape that carries what routing needs, per repository:

```JSON
{
  "version": 1,
  "rows": [
    { "page": "t3000/api-reference/data-points", "kind": "wrong", "note": "…", "at": "20
  ]
}
```

`page` is a page id, `at` is an ISO date, and `state` is `open` or anything else for handled. A row whose page is not in the registry is still reported — with its id, as plain text rather than a link — because that is a finding about the row, not a reason to drop it. With no such file, **Feedback by Owner** says the source is not agreed rather than reporting no feedback.

This shape is the extension's proposal, and the docs owner accepts or replaces it. It is the only part of the feedback design that is not settled.